

Part 1

A first class in my class

other / people in other class

A first class in my class

A second class of people but we are a first class

it of course is a world in our hands

up a school in my class

A school in my class (first) is a first class school

A school in my class is a first class school

A school in my class is a first class school

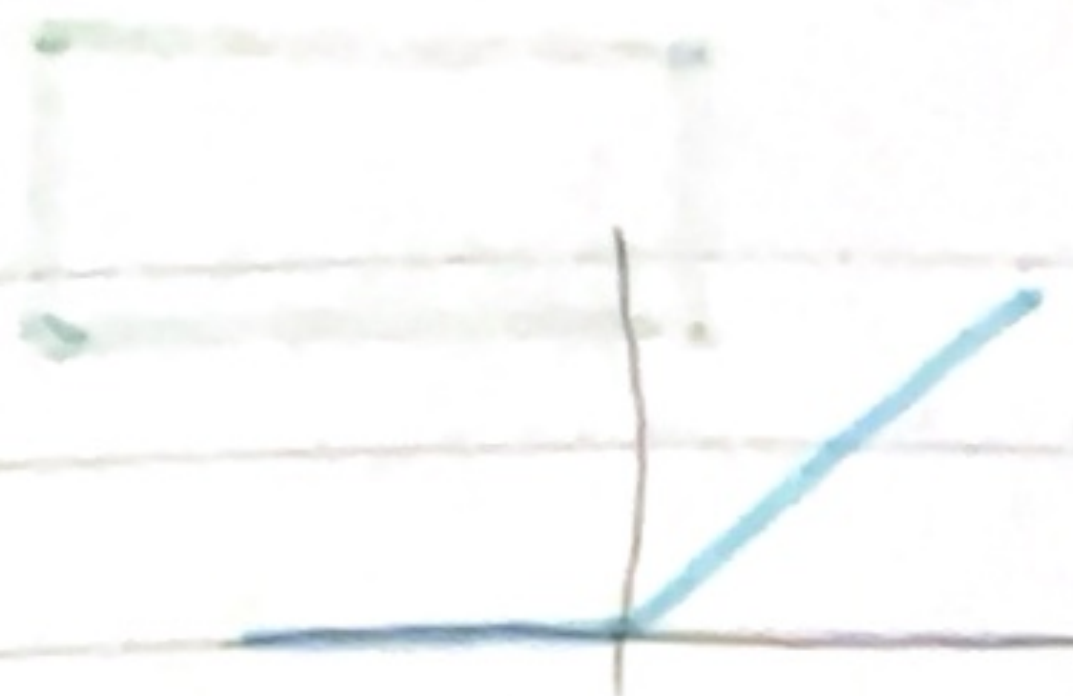
A school in my class is a first class school

A school in my class is a first class school

A school in my class is a first class school

A school in my class is a first class school

→ 27 ReLU → Rectified Linear Unit



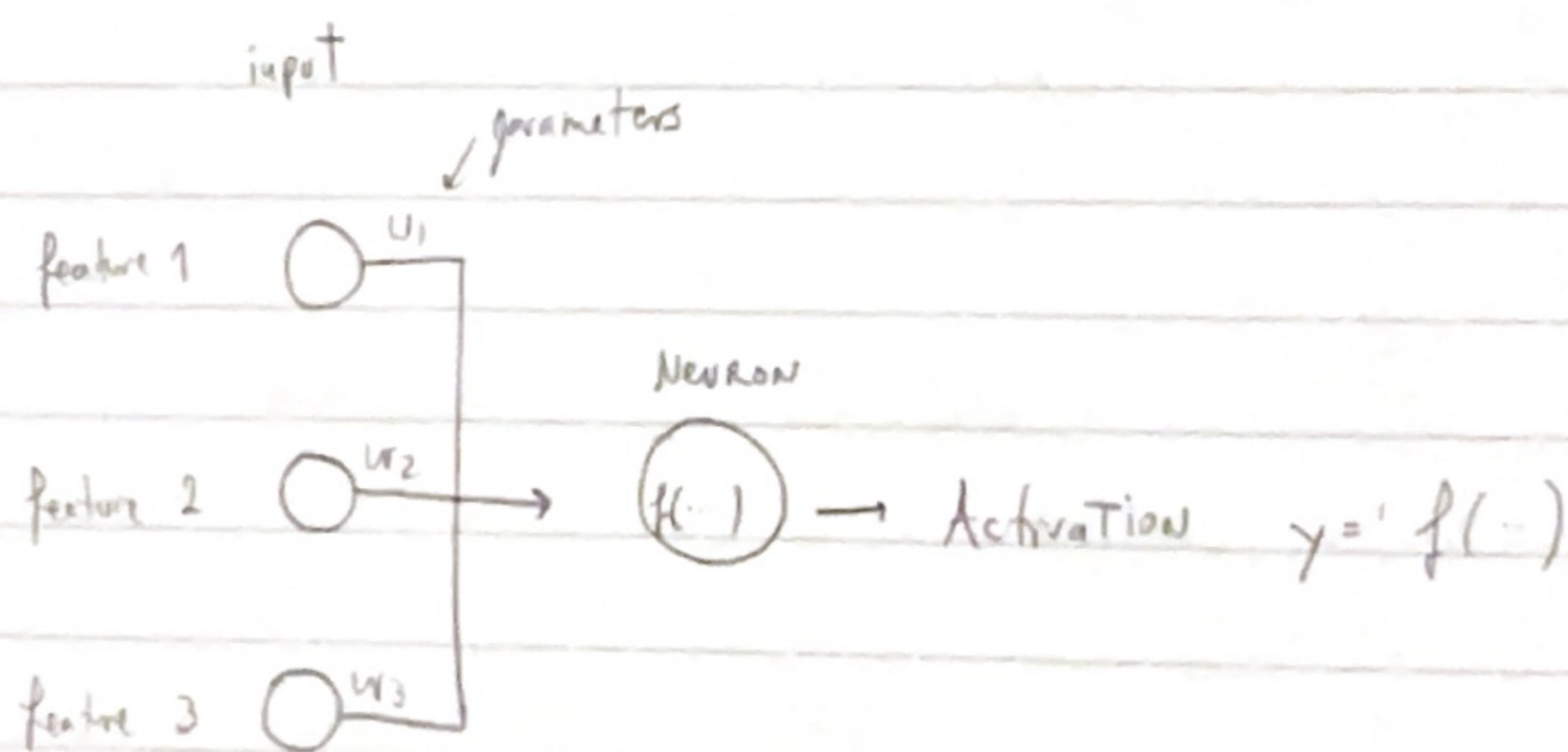
"rectifies" or passes through the positive values → which are linear
negatives don't pass → 0

not a line

→ Traditional machine learning methods → linear combination of input features
+/- non linear transformations

↓
produce activations

↓
predictions



layers of neurons → they all take the same input → produce an output

↓
Complex function

→ if all the neurons in the layer take as input all the input to the layer



fully connected layers = dense layer = linear layer

→ 2 fully connected layers in a neural network

1st layer
→ 2nd → model output

single neuron → represents binary classification probability that we want as output



Two-layer fully connected network

1st layer

linear
combination



non linear
transformation



first layer
activations

2nd layer

Result:

first layer
activations



linear
combination



non linear
transformation



number
(prediction)

→ Neural Architecture = how neurons are organized

going through training loop many times so the parameters are optimised

Training set



Validation set



Test set → No we see commercialised

⇒ optimisation step

→ Goal of training is to minimize Loss



the model calculates the loss



Adapts itself



Minimizes loss

⇒ Loss function → Mathematical $f(x)$ that processes all the individual losses



tells us how well a model performs

Types of Loss

depends on data
task requested to the model

- mean squared error loss (MSE)
- " " logarithmic error loss
- " absolute error loss (MAE)
- binary cross entropy
- hinge loss and squared hinge loss
- multiclass cross entropy loss
- sparse multiclass cross entropy loss

loss $f(x) =$

⇒ Unique properties → helps model in a particular way

→ $MSE = (\text{model prediction} - \text{ground truth})^2$

exponential
↗ bigger value
↘ no -

→ $MAE = |\text{model prediction} - \text{ground truth}|$

↗ values are linear
↘ no -

→ Cross entropy loss (= logistic loss / log loss)

classification problems → CATEGORIES
calculates the loss based on the probability of the predictions

negative log of error / natural log

allows more granular computation of error + closeness prediction to truth

GRADIENT DESCENT

→ it's an algorithm

⇒ How to adjust weights / biases?

→ which direction do we go to minimize loss?

Training a machine learning model = running the numerical optimisation algorithm to find the best weights for the data set and minimise the loss

Ex.

graphical look at gradient descent



→ this is every weight value for a function type

→ Most algorithms set the initial guess for the value of the weights as 0 or give a random number

Seeing point $\rightarrow$ figure out the derivative

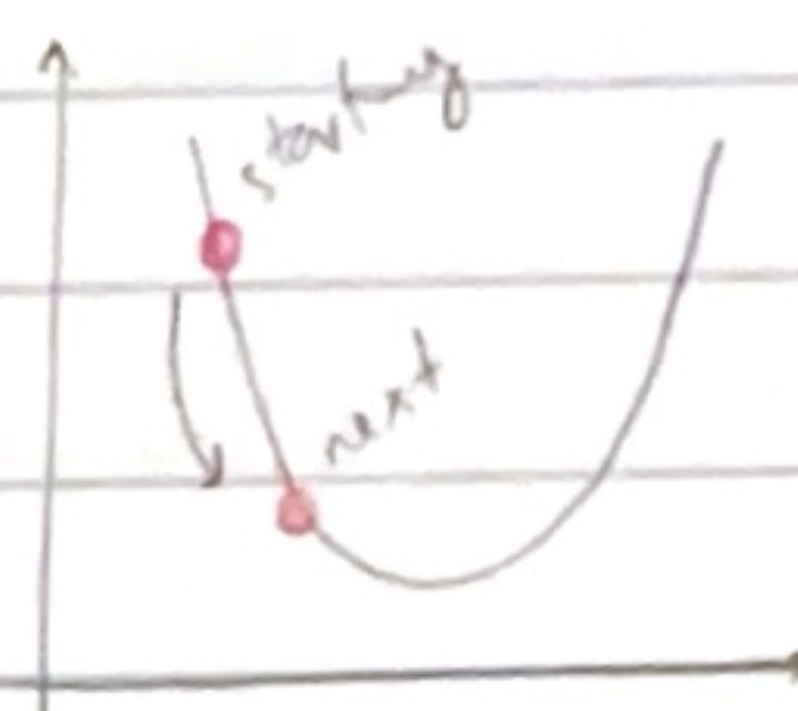
|
gradient $\rightarrow$ generalisation of idea of derivative / slope
that can be used for
functions of multiple variables
(parameters)

Backpropagation $\rightarrow$ technique to calculate gradients for complex $f(x)$

| and
simplify them into local gradient
computations
 $\downarrow$ that can
be combined together

parallel process makes it $\rightarrow$ possible
faster

Gradient $\rightarrow$ direction
magnitude
|



then the model takes a
step into the negative
gradient to
minimise loss

$\Rightarrow$ Represents itself aiming for the minimum

$\Rightarrow$ We can adjust the way we move towards minimal loss

adjusting $\rightarrow$ learning rate
step size

| can be adjusted by specialist

$\downarrow$
tuning hyperparameters

Representing unstructured image and text data

structured $\rightarrow$ there are
numbers and
"criteria"

unstructured $\rightarrow$ transform into numerical values
(the pixels)

Colored images $\rightarrow$ 3 numerical
values for
each pixel

$\rightarrow$ brightness of $\left\{ \begin{array}{l} \text{red} \\ \text{blue} \\ \text{green} \end{array} \right.$

$\downarrow$
then they are RESIZED

aim = single size + resolution

word embedding

Texts $\rightarrow$ represented mathematically

word embedding = geometrical / spatial representation
of a word

$\downarrow$
2 dimensional

$\downarrow$
need multidimensional (300 - 1024)

Types of Neural Networks

Convolutional Neural Networks (CNNs)

we need a layer that $\rightarrow$ one parameter efficient
 $\rightarrow$ preserve spatial information that's present in the pixel grids

$\downarrow$
convolutional layer $\rightarrow$ key building block is comprised of convolutional $\rightarrow$ filter
 $\rightarrow$ kernel

$\downarrow$
small group of parameters
(multiplied, summed)

the output of each linear combination

$\leftarrow$ a linear combination with one small patch or window of the image at a time

$\downarrow$
passed through an activation function (e.g. sigmoid $f(x)$)

$\rightarrow$ produces single value $\rightarrow$ placed relative to the location of the input patch in a new grid (something smaller)

$\rightarrow$ filter $\sim$ feature detector

$\rightarrow$ in a train model $\rightarrow$ high number $\rightarrow$ something of interest is found on the patch
 $\rightarrow$ low $\rightarrow$ nothing of interest

$\rightarrow$ each filter $\rightarrow$ scanning pixel grid, looking for patterns / objects of interest — one patch at the time

• receptive field = amount of the image that a given filter
has information for

• Many smaller kernel paradigms → instead of each parameter being
loaded into a single grid location

|
allows each filter - which is just a grid
| of weights
to move freely about the image

→ each filter aggregates info → at a global level

→ the model can leverage from the spatial info of
the image

NATURAL LANGUAGE PROCESSING (NLP)

⇒ ML approach to enable computers to understand and organize human languages

! Challenge is to understand the meaning
 ← computable
 ← interpretable
 ← accurate
 way

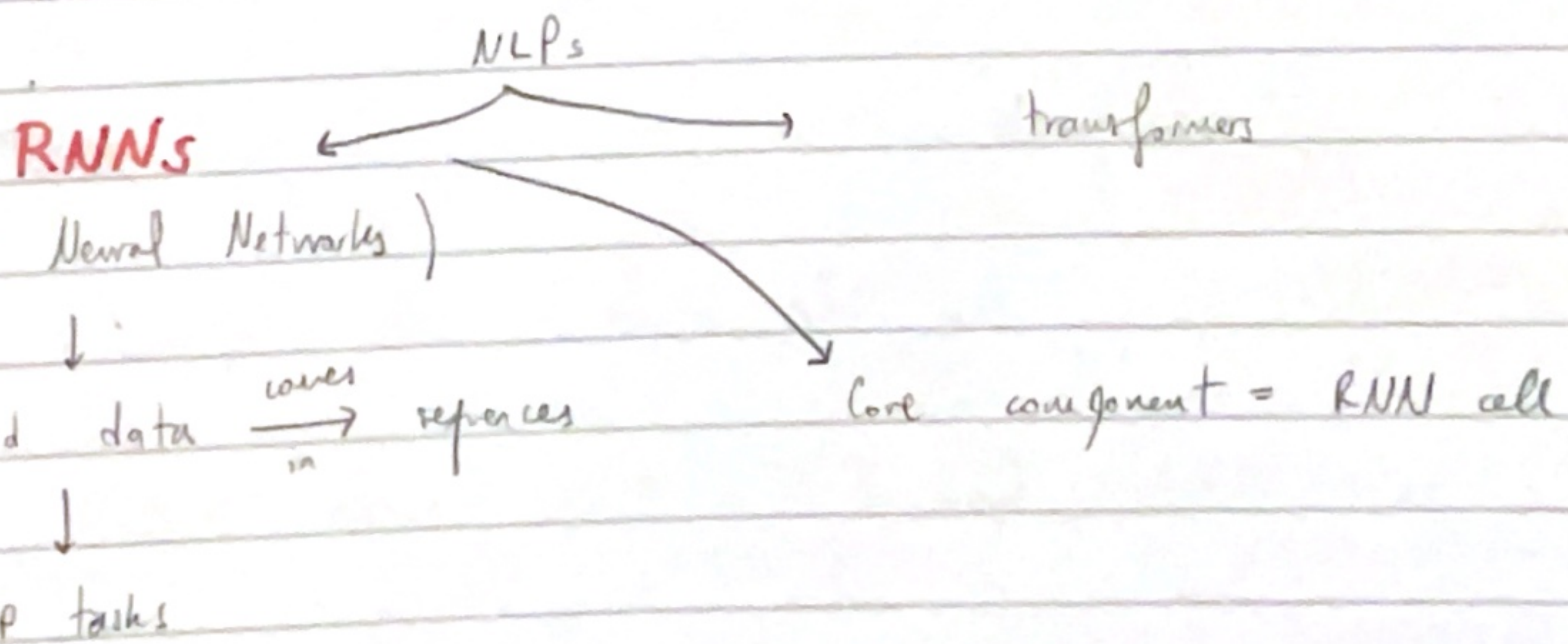
context } add needed info
interpretation
nuance }

⇒ Speech algorithms → focus on digitizing audio recordings → computable data

intonation } input
variable emphasis } meaning

human
 speed
 → then |
 into text

healthcare apps ← dictation
transcription



⇒ Recall words = embeddings or high dimensional coordinates → capture semantic meaning

Read 1 word at the time

⇒ RNN cells can take as input $\left\{ \begin{array}{l} \text{output from earlier layers of} \\ \text{neural network} \end{array} \right.$
($\neq$ to dense layers)
from before
they don't have
memory
additional context set of values
↓
can be used to pass info
from one time step to
the next

⇒ vector with "context" (no memory) → can learn to adjust values
(words already seen)

⇒ parameters to predict the final classifier probability → sometimes used in
the final step

⇒ $\neq$ variants of neural networks considering structure $\leftarrow \begin{array}{l} \text{LDSM} \\ \text{GRU} \end{array}$

⇒ RNNs are fundamental to sequence based problems $\left\{ \begin{array}{l} \text{DNA sequencing} \\ \text{dictation} \\ \text{motion tracking} \end{array} \right.$

⇒ Difficulties $\left\{ \begin{array}{l} \text{reading 1 word at the time} \rightarrow \text{grosser + messier} \\ \text{loss effectiveness in long sequences} \\ \text{technical = hard to parallelize processing of RNN} \\ \text{computations are sequentially} \end{array} \right.$
↓
 $0 \rightarrow 0$

TRANSFORMER

→ built on → self attention layer → processes in parallel while knowing the sequence

⇒ fundamental concept = the meaning of a word changes based on context

Self attention layers =

→ calculate weighting scores for the strength of contextual signal from any word to another

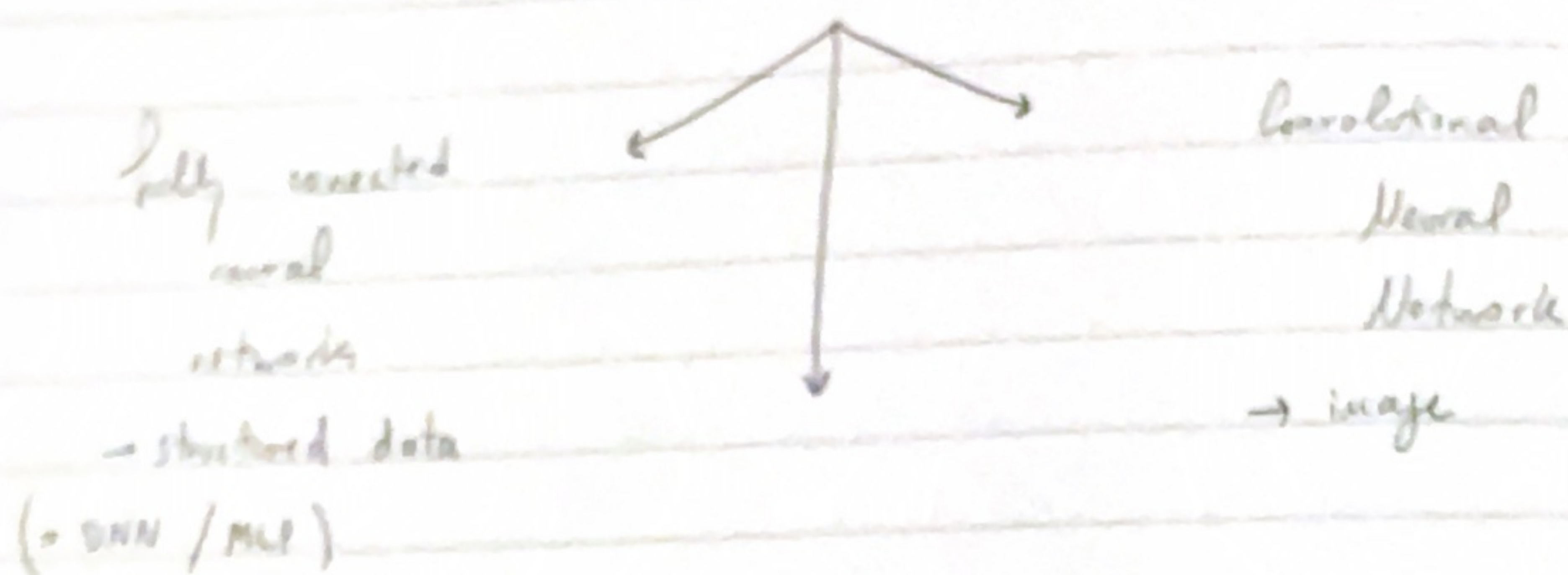
⇒ can produce $\neq$ weighted outputs → to encode $\neq$ types of context

→ transform inputs $\xrightarrow{\text{into}}$ outputs that → incorporate context at each element position in a variable way

⇒ Challenges → keeping positional information about how far away input elements are from each other

Summary

Major classes of Neural Network Architectures



→ Some other are hybrid

- Alexnet (2012) → conv → 8 layers
- VGG → 19 layers
- GoogleNet (2014) → 22
inception network
- ResNet (2015) Microsoft → 150 layers

→ SEGMENTATION → outputs one category prediction for each pixel

pixel mark

⊕ gives granularity

⊖ don't distinguish between distinct instances of a category object in the image (ex. tumors / lung nodules)

⇒ INSTANCE

SEGMENTATION

both

semantic segmentation

object detection

output

bounding box of each instance
+

pixel mark corresponding to the segmentation of the object within each bounding box

contain ≠ types of neural network architectures or structures to produce each type of output

→ classification branch → predicts category of objects

→ regression branch → outputs numerical values

location
extent

} of object
bounding box

allow multiple classification regression outputs

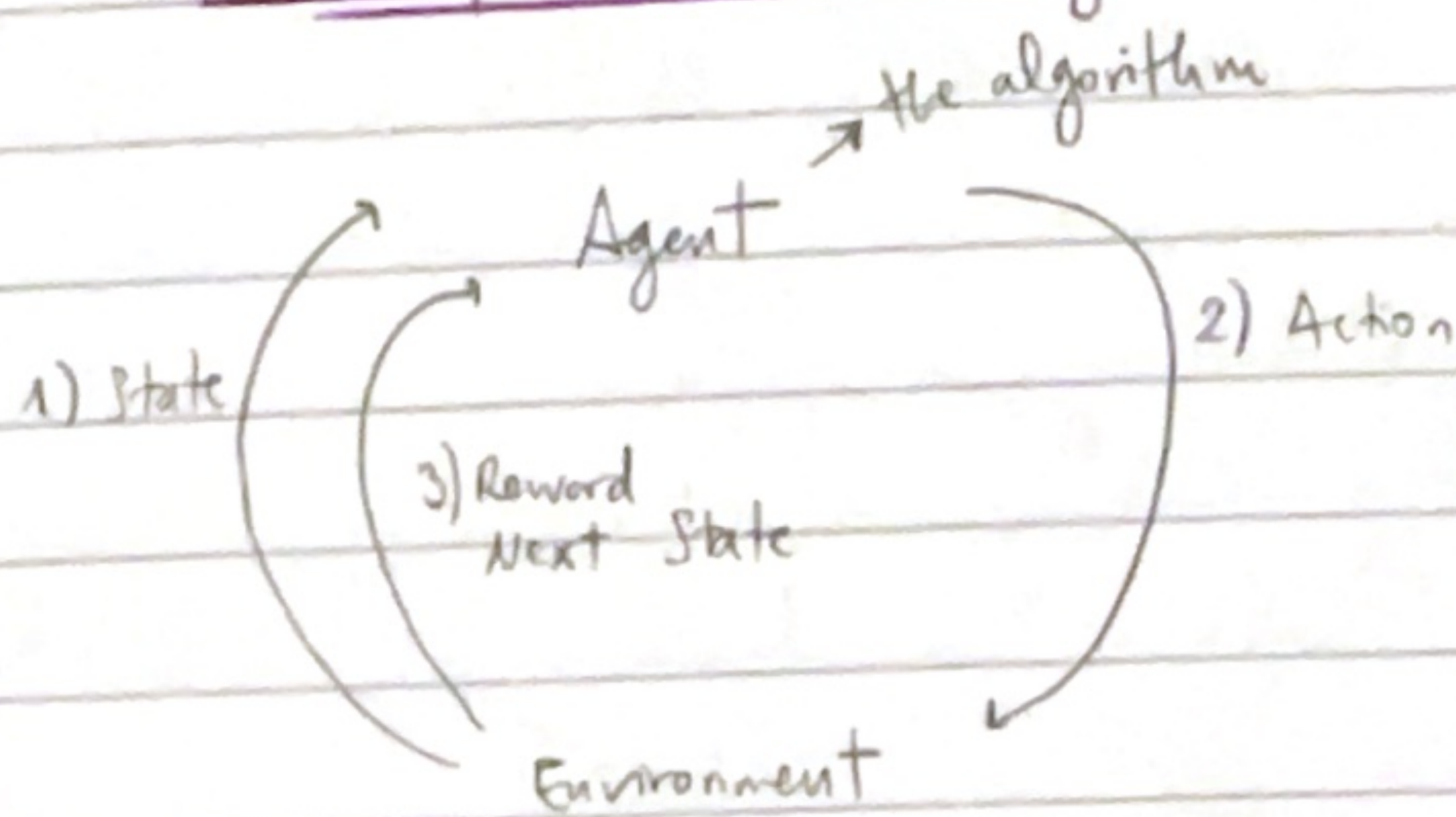
corresponding to ↓

multiple objects that can be detected in an image

Detection → ex. individual cells

Reinforcement learning

→ e.g. Gaming
trial + error



Data in medicine is:

inferential
discontinuous } which makes it difficult

learning accurate functions $\xrightarrow{H_t}$ map from desired inputs
↓ to
outputs
| from

features describing the environment
observation to a value or
"reward"

of how good any particular action is given the observation features ← (based on minimising a loss function via gradient descent)